DataLake Big Data Analytics Cloud Documentation

Industry: Big Data & Analytics Platform

Case Study: 114 : DataLake Big Data Analytics Cloud

Student Details:

- **Student Name:** Rayan Rawat
- **Cohort:** Mark Zuckerberg
- **Roll No:** 150096724141
- **Subject:** AWS

Live Link:-

https://datalake-aws-project.onrender.com/

Git-Hub Link:-

https://github.com/Rayan-141/DataLake_BigData_AWS_Project

Documentation Link:-

https://docs.google.com/document/d/1ZoXt8zKwgQOUTIwok3BEZRo8q-e9JmOkTESks6rCYaw/edit?usp=sharing

Problem Statement:-

The **DataLake Big Data Analytics Cloud** project addresses the challenges of managing datasets using traditional local systems. The existing system relies on local file storage, manual data management, limited monitoring, and weak security, making it difficult to manage large datasets efficiently.

The organization requires a centralized cloud platform that provides secure file storage, centralized data management, infrastructure monitoring, user access control, alerts, and a web-based dashboard accessible from anywhere.

Existing System:-

- Files are stored on local computers.
- Data is managed using spreadsheets and manual processes.
- Reports are created manually.
- There is no centralized dashboard.
- No proper monitoring of server health.
- No alert system for resource issues.
- Data sharing is done manually.
- Security is limited and cloud access control is not available.
- The local database has no centralized cloud storage.

Problems in Existing System:-

1. **Local File Storage:** Files can be lost if the computer crashes.
2. **No Centralized Dashboard:** Administrators cannot manage everything from one place.
3. **No Monitoring:** CPU, Memory, Disk, and Network usage cannot be monitored.
4. **No Alerts:** Administrators are not notified when server resources become high.
5. **Limited Security:** No centralized user and permission management.
6. **Manual Dataset Management:** Uploading, downloading, and managing datasets is difficult.
7. **Limited Accessibility:** The application cannot be accessed remotely.
8. **Manual Deployment:** Deploying the application requires manual setup on every system.

Proposed Solution:-

The proposed solution is to build a centralized **DataLake Big Data Analytics Cloud Platform** using AWS services and a web-based dashboard. The platform securely stores datasets, monitors infrastructure, manages users, and provides centralized access through a live cloud application.

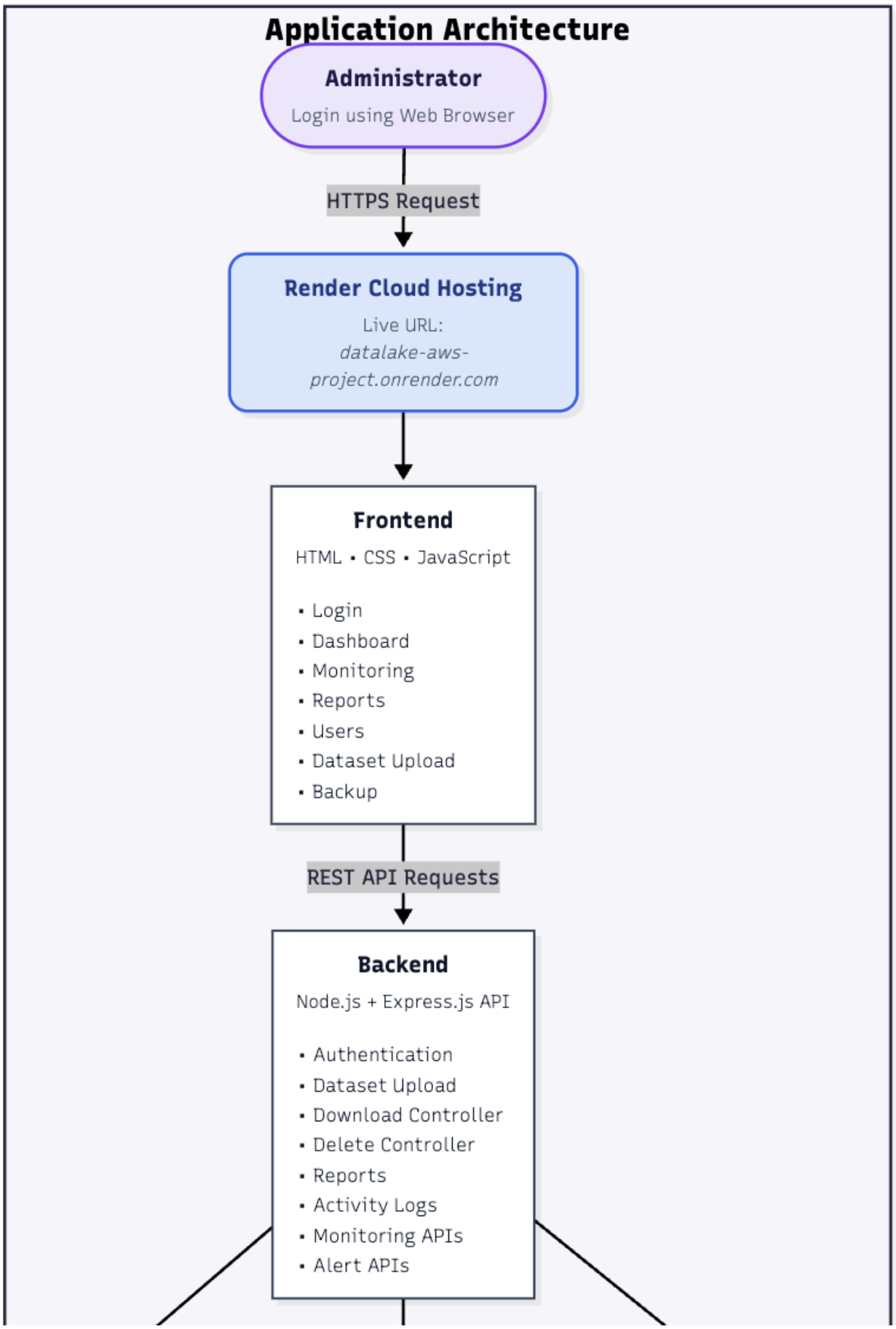
Solution Highlights:

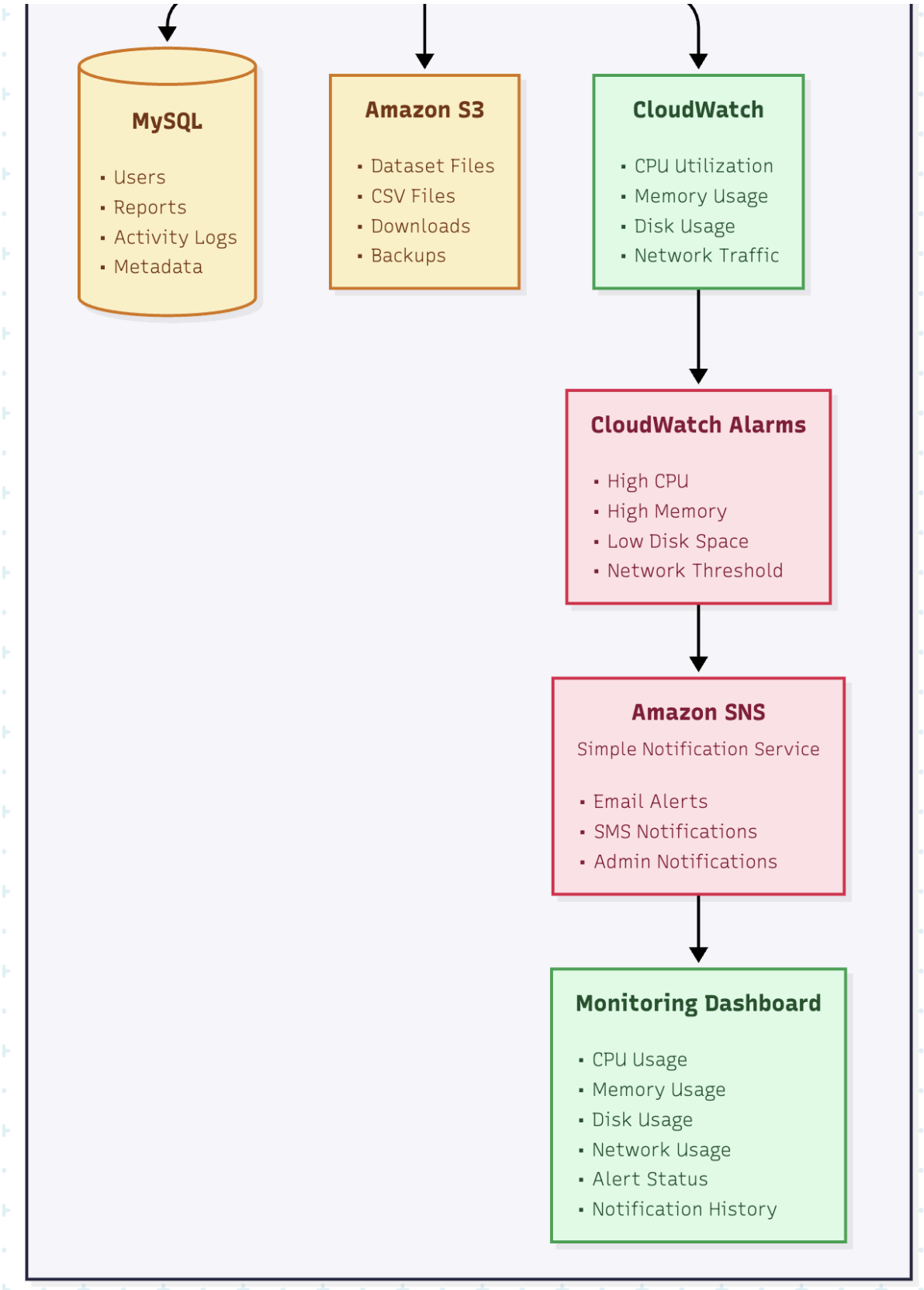
- **Amazon S3** – Secure cloud storage for uploaded datasets.
- **AWS IAM** – Secure access and permission management.
- **MySQL** – Centralized database for users, reports, and dataset metadata.
- **Amazon CloudWatch** – Infrastructure monitoring (CPU, Memory, Disk, Network).
- **CloudWatch Alarms** – Detects high resource usage and generates alerts.
- **Amazon SNS** – Sends alert notifications to administrators.
- **Docker** – Containerized application deployment.
- **GitHub** – Source code management and version control.
- **Render** – Live cloud hosting for the web application.
- **Web Dashboard** – Centralized management for datasets, monitoring, reports, users, and infrastructure status.

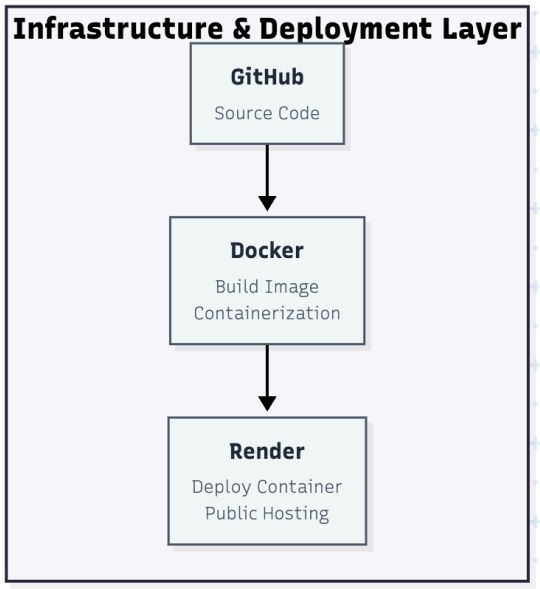
Working Flow:- image link

⇩

AWS DataLake – Big Data Analytics Cloud (System Architecture)




MySQL
Users
Reports
Activity Logs
Metadata
Amazon S3
Dataset Files
CSV Files
Downloads
Backups
CloudWatch
CPU Utilization
Memory Usage
Disk Usage
Network Traffic
CloudWatch Alarms
High CPU
High Memory
Low Disk Space
Network Threshold
Amazon SNS
Simple Notification Service
Email Alerts
SMS Notifications
Admin Notifications
Monitoring Dashboard
CPU Usage
Memory Usage
Disk Usage
Network Usage
Alert Status
Notification History


Infrastructure & Deployment Layer
GitHub
Source Code
Docker
Build Image
Containerization
Render
Deploy Container
Public Hosting


AWS Cloud Services
Amazon EC2
(Infrastructure)
• Virtual Cloud Server
• Backend Hosting Environment
• Compute Resources
Amazon S3
• Dataset Storage
• CSV Upload
• Backup Storage
• Secure Object Storage
AWS IAM
• User Authentication
• Role Management
• Secure AWS Access
• Permission Control
Amazon CloudWatch
• CPU Monitoring
• Memory Monitoring
• Disk Monitoring
• Network Monitoring
• Performance Metrics
CloudWatch Alarms
• High CPU Alert
• Memory Threshold
• Disk Space Alert
• Network Alert
Amazon SNS
• Email Notifications
• SMS Alerts
• Administrator Notifications
• Alert Distribution

Objectives:-

- **Centralized Platform:** A single UI to manage all data at https://datalake-aws-project.onrender.com/
- **Secure Authentication:** Implementing JWT or session-based login for role-based access.
- **Elastic Storage:** Using S3 for scalable file management.
- **Monitor:** CPU, Memory, Disk, and Network utilization.
- **Real-time alerts:** Alerts for abnormal resource usage.

AWS Services Used:-

1. Amazon EC2 (Elastic Compute Cloud)

Why Used?

- To host the application on a cloud server.

Where Used?

- Backend server.

Purpose

- Runs the Node.js application in the cloud instead of a local machine.

2. Amazon S3 (Simple Storage Service)

Why Used?

- To store uploaded datasets.

Where Used?

- Dataset Upload section.

Purpose

- Keeps files safe, secure, and easily accessible.

3. AWS IAM (Identity & Access Management)

Why Used?

- To secure AWS resources.

Where Used?

- AWS Account.

Purpose

- Controls who can access AWS services and what actions they can perform.

4. Amazon CloudWatch (Simulated)

Why Used?

- To monitor infrastructure health.

Where Used?

- Monitoring Dashboard.

Purpose

- Shows server metrics like:
    - CPU Usage
    - Memory Usage
    - Disk Usage
    - Network Usage

5. Amazon CloudWatch Alarms

Why Used?

- To detect abnormal resource usage and trigger alerts..

Where Used?

- Monitoring & Alerting System.

Purpose

- Displays alerts when CPU, Memory, Disk, or Network usage becomes high.

5. Amazon SNS (Simple Notification Service)

Why Used?

- To send alert notifications to administrators.

Where Used?

- Alert Notification System.

Purpose

- Delivers Email Notifications , SMS Notifications , Administrator Alerts , etc

Technology Stack:

Layer	Technology
Frontend	HTML5, CSS3, JavaScript (Nginx Web Server)
Backend	Node.js, Express.js Framework
Database	MySQL (Relational)
Storage	Amazon S3
Compute	AWS EC2 (Ubuntu)
Container	Docker & Docker Hub
Control	Git & GitHub

Architecture:-

The architecture follows a standard 2-tier web pattern:

1. **User** hits the **Elastic IP** (44.208.74.131).
2. **Nginx** acts as a reverse proxy, forwarding traffic to **Node.js** on port 3000.
3. **Node.js** processes logic and queries the **MySQL** database for records.
4. **S3** handles file storage via AWS SDK.
5. **CloudWatch** monitors the EC2 instance and sends alerts via **SNS**.

Problem Statement vs Proposed Solution :-

Previously (Existing System)	Now (Proposed Solution)
Files were stored on local desktops.	Files are securely stored in **Amazon S3**.
Application depended on a single local system.	Application is hosted on a **cloud server (EC2/Cloud Infrastructure)**.
No centralized dashboard for management.	A **Web Dashboard** manages datasets and monitoring from one place.
No proper monitoring of server resources.	**CloudWatch** monitors CPU, Memory, Disk, and Network usage.
No alert system for server issues.	**CloudWatch Alarms** generate alerts for resource issues.
Administrators were not notified about problems.	**Amazon SNS** sends alert notifications to administrators.
No secure access to cloud resources.	**AWS IAM** provides secure user access and permissions.
Dataset information was difficult to manage.	**MySQL Database** stores dataset details and application data.
Deployment was manual and time-consuming.	**Docker** simplifies application deployment.
Application could not be accessed from anywhere.	**Render** hosts the application with a live public URL.

Problem → Solution Flow:-

Local File Storage

|

▼

Amazon S3 Storage

No Monitoring

|

▼

CloudWatch Monitoring

No Alerts

|

▼

CloudWatch Alarms

No Notifications

|

▼

Amazon SNS

No Secure Access

|

▼

AWS IAM

No Central Dashboard

|

▼

Web Dashboard

Manual Deployment

|

▼

Docker + Render

Scattered Data

|

▼

MySQL Database

Limitations:-

- Uses **MySQL** instead of **Amazon RDS**.
- **CloudWatch** metrics are simulated for demonstration.
- **CloudWatch Alarms** and **Amazon SNS** are implemented as project concepts, not full real-time integration.
- Does not include **Auto Scaling** or **Load Balancer**.
- Supports a **single-region deployment** only.
- Advanced enterprise security features are not implemented.

Conclusion:-

The **DataLake Big Data Analytics Cloud** project provides a secure and centralized cloud platform for managing datasets. It uses **Amazon S3** for storage, **AWS IAM** for security, **CloudWatch** for monitoring, **CloudWatch Alarms** and **Amazon SNS** for alerting concepts, and **Docker** and **Render** for deployment. The project improves data storage, monitoring, security, and accessibility, while providing a strong foundation for future cloud enhancements.